

# AI-Assisted Software Development

## Module 3: Development Workflow with AI Tools

Mike Burton

# Overview

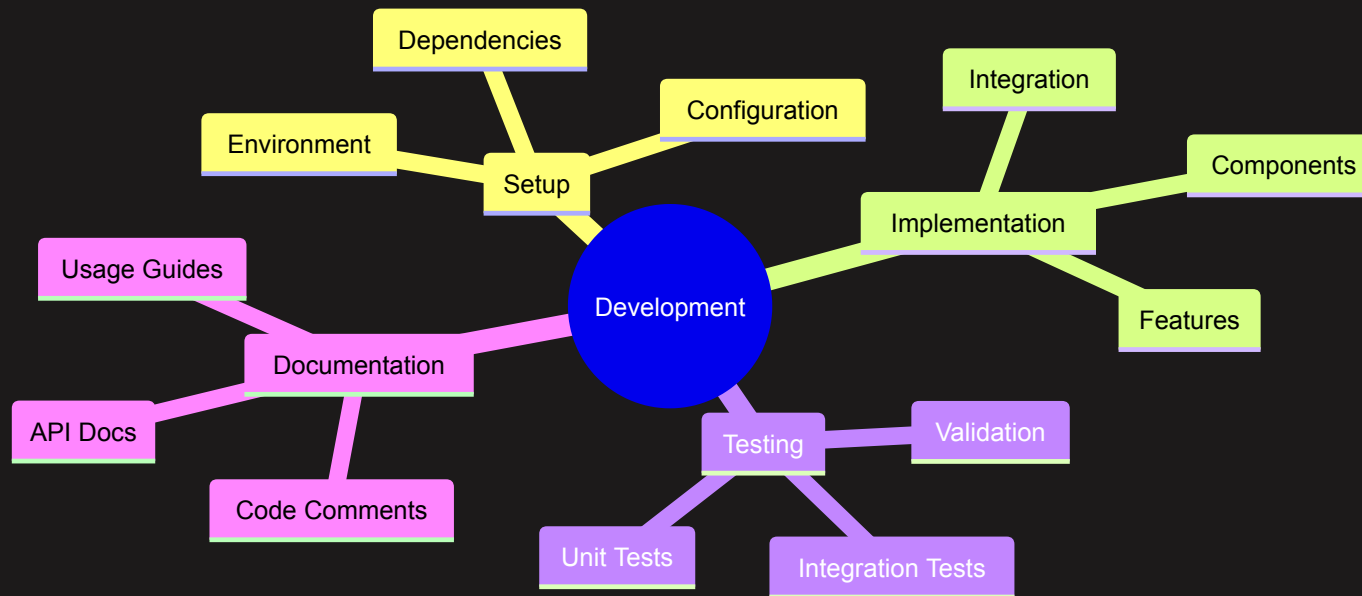
What we'll cover:

- Project setup
- Code generation
- Testing strategies
- Debugging approaches
- Documentation

Tools we'll use:

- Cursor for development
- AI for code generation
- AI for debugging
- AI for testing

# The Development Process



# Project Setup with AI

## 1. Initialize Project Structure

```
Ask AI to generate:  
- Project scaffolding  
- Configuration files  
- Dependencies list  
- Build scripts
```

## 2. Configure Development Environment

```
Set up:  
- Development tools  
- Linting rules  
- Testing framework  
- Build pipeline
```

# Working with Cursor

Maximize AI assistance:

## 1. Context Management

- Keep related files open
- Organize code logically
- Use clear file names
- Maintain documentation

## 2. Effective Prompting

- Be specific about requirements
- Reference existing code
- Specify error handling
- Include edge cases

# Code Generation Strategies

## 1. Component Creation

Prompt Structure:

- Component purpose
- Required props
- Expected behavior
- Error handling
- Edge cases

Example Prompt:

Create a TaskList component that:

- Accepts tasks array
- Supports filtering
- Handles loading states
- Implements pagination
- Manages error states

## 2. Feature Implementation

Include:

- User requirements
- Technical constraints
- Integration points
- Performance needs

# AI-Assisted Testing

## 1. Test Case Generation

Generate test cases for:

- Happy paths
- Edge cases
- Error conditions
- Performance scenarios

## 2. Test Implementation

```
describe('TaskList', () => {  
  it('renders tasks correctly')  
  it('handles empty state')  
  it('manages loading state')  
  it('handles errors')  
  it('supports pagination')  
})
```

# Debugging with AI

Effective debugging workflow:

## 1. Error Analysis

- Share error message
- Include stack trace
- Provide context
- Describe expected behavior

## 2. Solution Generation

- Get multiple approaches
- Evaluate trade-offs
- Test solutions
- Document fixes



# Code Review Process

Use AI for code review:

## 1. Quality Checks

- Code style
- Best practices
- Performance
- Security

## 2. Documentation Review

- Completeness
- Clarity
- Examples
- Edge cases

# Integration Workflow

Managing feature integration:

## 1. Pre-Integration

- Code review
- Test coverage
- Documentation
- Performance check

## 2. Integration Steps

- Merge strategy
- Conflict resolution
- Testing
- Deployment

# Documentation During Development

Maintain living documentation:

## 1. Code Documentation

```
/**  
 * TaskList Component  
 * @param {Task[]} tasks – Array of task objects  
 * @param {Object} filters – Active filters  
 * @param {Function} onUpdate – Update handler  
 * @returns {JSX.Element} Rendered task list  
 */
```

## 2. Implementation Notes

```
## Feature: Task Management  
– Implementation details  
– Design decisions  
– Trade-offs made  
– Future considerations
```

# Performance Optimization

Use AI for optimization:

## 1. Analysis

- Identify bottlenecks
- Profile performance
- Memory usage
- Network calls

## 2. Improvements

- Code optimization
- Caching strategies
- Lazy loading
- Resource management

# Error Handling

Implement robust error handling:

```
try {  
    // Operation code  
} catch (error) {  
    // Error classification  
    if (error instanceof NetworkError) {  
        // Handle network issues  
    } else if (error instanceof ValidationError) {  
        // Handle validation failures  
    } else {  
        // Handle unexpected errors  
    }  
}
```

# Security Considerations

Use AI to implement security:

## 1. Input Validation

```
function validateInput(data) {  
  // Sanitize input  
  // Check types  
  // Validate format  
  // Prevent injection  
}
```

## 2. Authentication

```
function authenticateUser(credentials) {  
  // Verify credentials  
  // Generate tokens  
  // Set session  
  // Log activity  
}
```

# Best Practices

## 1. Code Quality

- Clear naming
- Consistent style
- Proper documentation
- Error handling

## 2. Development Process

- Regular commits
- Meaningful messages
- Branch management
- Code review

## 3. Testing

- Unit tests
- Integration tests
- Edge cases
- Performance tests

# Common Pitfalls

## 1. Development

- ❌ Skipping documentation
- ❌ Ignoring edge cases
- ✅ Regular documentation
- ✅ Comprehensive testing

## 2. AI Usage

- ❌ Blindly accepting output
- ❌ Unclear prompts
- ✅ Verify solutions
- ✅ Clear requirements



# Recap

- Set up project properly
- Use AI effectively for development
- Implement comprehensive testing
- Maintain documentation
- Focus on code quality
- Consider security
- Optimize performance
- Handle errors properly

## Next Steps

- Review implementation
- Run performance tests
- Complete documentation
- Plan deployment